PROJECT 1

Field Sales & Follow-Up Management System

Field Employee Sales & Follow-Up Management Platform

Internal Client Engagement — Client Identity Withheld

BHARAT INFOTECHS

Software Product Requirement & Development Handbook

Version 1.0 — Initial Project Brief

Confidentiality Notice

This document represents the initial project understanding available at the time of preparation. Requirements may be updated following client confirmation. Teams must not assume that proposed features are final requirements until approved.

Table of Contents

1. Project Overview

CONFIRMED / INITIAL REQUIREMENT

Bharat Infotechs is engaged to design and build a **Field Sales & Follow-Up Management System** for a client organisation that operates a distributed team of field employees. The client's business name is intentionally withheld from this handbook; the team should treat this as a real production engagement and not a hypothetical exercise.

Field employees visit customer sites to record sales activity, material supply, and follow-up information. Today this happens largely off-system (paper, spreadsheets, or messaging apps); the goal of this platform is to centralise that activity into a single, auditable, searchable system that both the field team and the management/admin team can rely on.

Who is involved

- **Field Employees** — create field/follow-up records from the location of the visit.
- **Administrators / Back-office staff** — monitor, search, filter, and export the records created by field employees, and manage the employee roster.
- **Bharat Infotechs development team (you)** — responsible for turning this brief into a working, tested, deployed application.

2. Problem Statement

CONFIRMED / INITIAL REQUIREMENT

Field employees currently have no unified digital way to record a site visit, the material supplied, the customer/site involved, and the follow-up required next. This makes it difficult for management to know, in near real time: who visited which site, when, what was supplied, whether a follow-up is pending, and where the record was created from (location proof). Reporting for internal review or for a company-standard export currently depends on manual collation.

3. Objectives

- Give every field employee a simple, fast way to log a visit/follow-up record, from a phone, including photographic and location proof.
- Centralise all field records in one searchable database instead of scattered paper/chat records.
- Give administrators full visibility: search, filter, view, and export records.
- Support a company-standard export format for downstream reporting (format to be confirmed).
- Build the system so it can scale to more employees, more record types, and more reporting needs without a redesign.

4. Target Users

User	Description	Primary Device
Field Employee	Visits customer sites; creates field/follow-up entries in real time or shortly after a visit.	Mobile phone (Android likely primary — confirm with client)
Administrator	Back-office role. Manages employees, reviews all field records, exports reports.	Desktop / laptop browser
Super Admin (proposed)	Highest privilege; manages admin accounts and system configuration. To be confirmed if required.	Desktop / laptop browser

5. Functional Requirements

CONFIRMED / INITIAL REQUIREMENT

Field Employee side

- Login using a unique Employee ID and password/PIN.
- View a personal dashboard (recent entries, pending follow-ups).
- Create a new field/follow-up record capturing: customer/site, GPS location, a site image, material supplied, quantity, follow-up notes, date/time and remarks.
- Record is timestamped automatically at creation; the employee should not be able to backdate it unless the client explicitly requires that (mark as proposed/needs confirmation).
- View own submitted history.

Administrator side

- Manage the employee roster (add, edit, deactivate employees; not necessarily hard-delete).
- View all field records with search and multi-field filtering (employee, date range, customer/site, material).
- Open a single record to see full detail: images, GPS location on a map, material and quantity, follow-up notes, submission timestamp.
- Generate and download/export reports in the company template (template TBC).
- View basic operational reports (e.g. entries per employee, per period — exact KPIs proposed, to confirm with client).

PROPOSED / TO BE CONFIRMED

Whether employees can edit/delete their own submitted records after creation is not yet defined by the client and should be treated as a clarification question (see Section 33).

6. Non-Functional Requirements

Category	Requirement
Performance	Record submission (including image upload) should complete within a few seconds on a typical mobile network; list/search views should paginate rather than loading all records at once.
Availability	Central data store should be durable and backed up; field employees may need an offline-friendly capture flow if connectivity at sites is unreliable (proposed — confirm with client).
Usability	Field employee flow must be usable one-handed, in the field, with minimal typing.

Category	Requirement
Scalability	Must support growth in employee count and record volume without architectural rework.
Security	Role-based access, encrypted transport (HTTPS), and secure storage of images/location data.
Auditability	Every record must be traceable to the employee, device/session, and timestamp that created it.
Maintainability	Codebase should follow the SDLC, branching, and documentation practices in this handbook.

7. User Roles & Access

Role	Access Summary
Field Employee	Create/view own field records only. No access to other employees' records or admin functions.
Administrator	Full read access to all field records, employee management, export/reporting.
Super Admin (proposed)	Administrator access plus system/user configuration — confirm if this tier is required.

8. User Journeys

8.1 Field Employee — creating a follow-up record

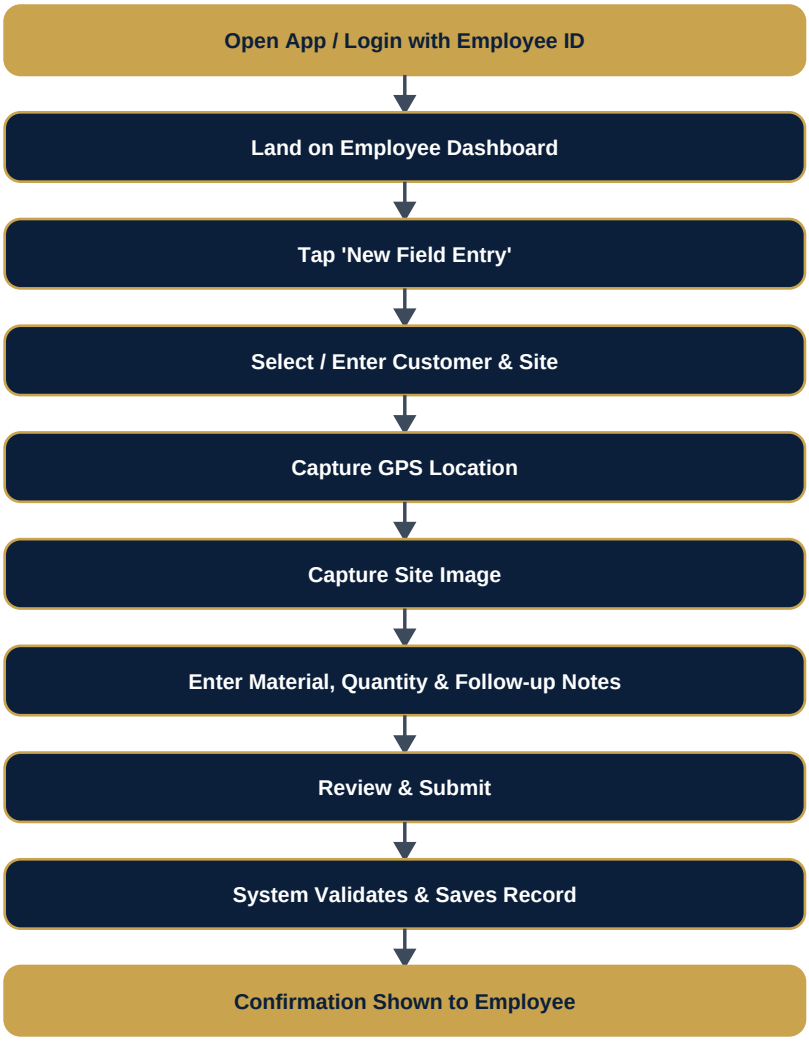

Fig 8.1 — Field employee record-creation journey

8.2 Administrator — reviewing and exporting records

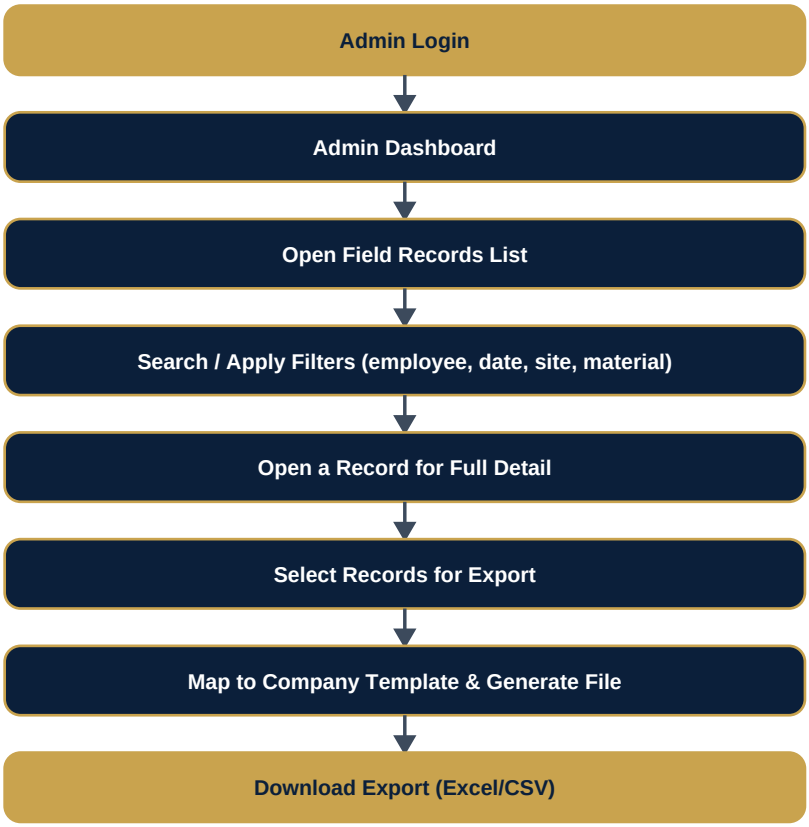

Fig 8.2 — Administrator review & export journey

9. Feature / Module Breakdown

Module	Key Features
Authentication	Employee ID + password/PIN login; admin login; session handling; role-based redirect.
Field Entry Module	Create field record, GPS capture, image capture, material/quantity entry, follow-up notes, validation, save/confirmation.
Employee Management (Admin)	Add/edit/deactivate employee, assign Employee ID, view employee profile & history.
Records Management (Admin)	List, search, filter, view-detail, and status tracking of all field records.
Reporting & Export	Filtered export to company template, download as Excel/CSV, basic operational reports.
Location Services	Permission handling, capture, accuracy handling, map display of a record's location.
Media Handling	Image capture/upload, compression, storage, association with a record.
Notifications (proposed)	Optional reminders for pending follow-ups — to be confirmed as in/out of scope.

10. Employee ID System

PROPOSED / TO BE CONFIRMED

A structured, human-readable Employee ID makes records easy to trace and is proposed in the following format. The final format and any client-mandated prefix must be confirmed before development locks the schema.

Proposed format: SE-FS-001, SE-FS-002, SE-FS-003 … where *SE* denotes the client's internal code (placeholder), *FS* denotes the Field Sales module, and the numeric suffix is a zero-padded sequential value, unique and never reused even if an employee is deactivated.

- Employee ID is generated once, at employee onboarding, by the Administrator.
- Every field record stores the creating employee's ID as a foreign key — this is how every record is traced back to a person, independent of the employee's display name (which could change).
- Employee ID should never be reused after deactivation, to preserve the integrity of historical records.
- Login should accept the Employee ID (not necessarily an email), since field employees may not have personal email addresses — confirm with client.

11. GPS & Location Capture

11.1 Behaviour

- Location is requested and captured only at the moment a field record is created (point-in-time capture), not continuous background tracking, unless the client specifically requires live tracking (treat as a clarification question).
- Capture latitude, longitude, an accuracy/precision value, and a timestamp alongside the record.
- If GPS permission is denied, the employee should be shown a clear explanation and given the option to retry, with the record blocked from submission until location is available or an explicit manual override is agreed with the client.
- If location is unavailable (e.g. indoors, poor signal), retry with a timeout and allow the employee to proceed only if the client agrees a location-optional fallback is acceptable.

11.2 Security & privacy considerations

- Treat GPS coordinates as personal/sensitive data — encrypt in transit (HTTPS) and restrict read access to Administrators only.
- Consider basic spoofing awareness (e.g. flagging mock-location settings on Android) if the client considers location-proof integrity important; this is a proposed hardening step, not a baseline requirement.
- Location data should be used strictly for the purpose of verifying field-visit records — do not repurpose it for continuous employee monitoring unless the client has explicitly requested and consented to that scope.

12. Image Capture

- Employee captures a photo directly via device camera at the time of record creation, or uploads an existing image if camera capture is not possible.
- Compress images client-side before upload to keep upload time and storage cost reasonable.

- Use a collision-safe file naming convention, e.g. {employeeId}_{recordId}_{timestamp}.jpg.
- Store images in a dedicated object/file storage layer and keep only a reference (URL/path) in the database record, not the binary itself.
- On upload failure, retry with the record saved in a 'pending image' state rather than losing the whole submission.
- Restrict image access to authenticated Administrators and the owning employee.

13. Export System

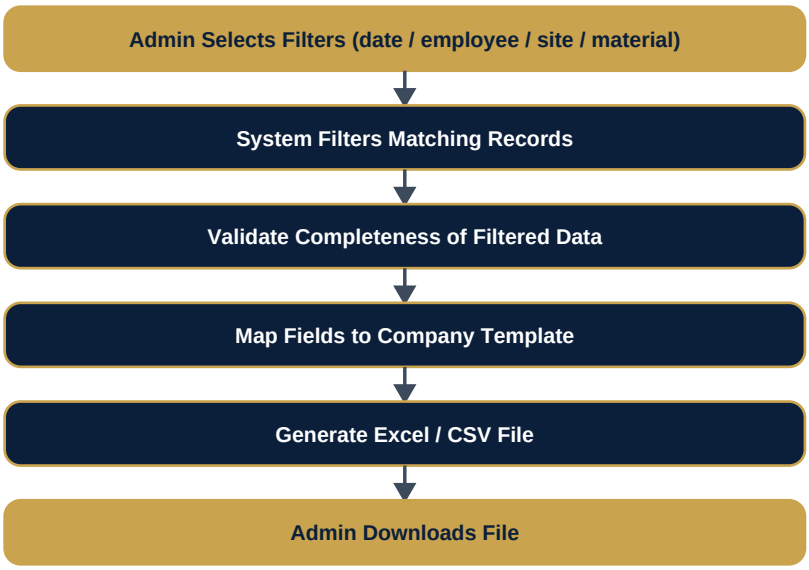

Fig 13.1 — Export workflow

PROPOSED / TO BE CONFIRMED

The exact column layout, header names, and file format of the 'company template' referenced by the client have not yet been supplied. Build the export module with a configurable field-mapping layer so the template can be finalised without a structural rewrite.

14. SDLC Process

Fig 14.1 — SDLC stages the team must follow for this project

15. Requirement Analysis Approach

Before writing code, the team should turn this brief into a concrete backlog: restate every functional requirement as a user story, mark each as CONFIRMED or PROPOSED, and list the assumptions being made for every PROPOSED item. Any assumption that materially affects the data model or the export template should be raised with the project coordinator before development of that area begins.

- Convert Section 5 requirements into a prioritised user-story backlog.
- Identify which PROPOSED items block schema/API design and flag them early.
- Produce the Team Planning Exercise deliverables (Section 34) before major development starts.

16. System Architecture

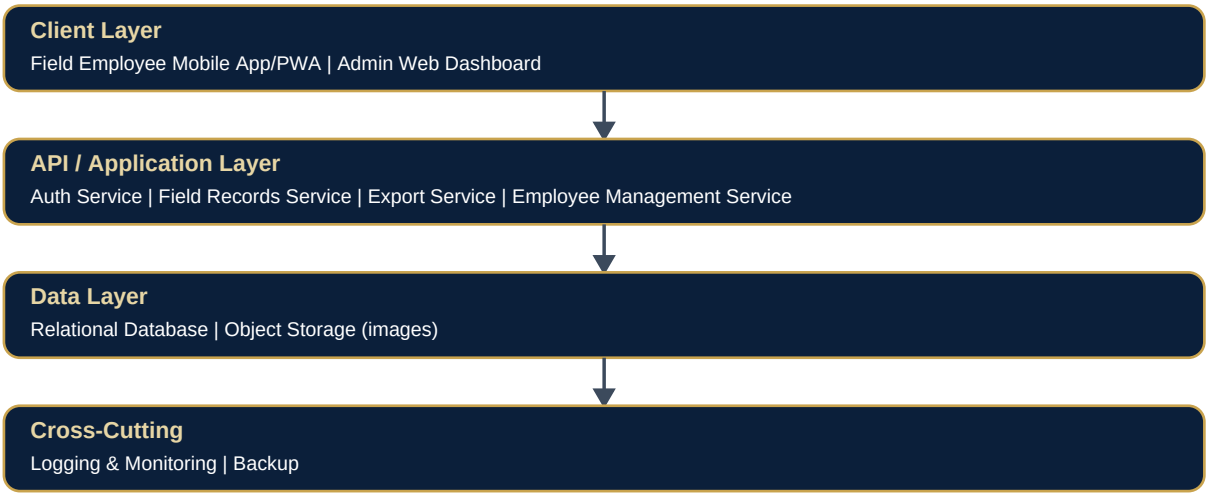

Fig 16.1 — High-level layered architecture (indicative)

17. Suggested Architecture Options

PROPOSED / TO BE CONFIRMED

Option	Description	When it fits
Monolithic web app + mobile-responsive UI (PWA)	Single backend serving both a mobile-friendly web app for employees and a dashboard for admins.	Fastest to build for an internship timeline; recommended default unless the client wants a native app.
Backend API + separate native mobile app	REST/GraphQL API consumed by a dedicated Android/iOS app for employees and a separate web dashboard for admins.	If the client specifically wants an installable native app with offline capture.
Backend API + cross-platform app (e.g. Flutter/React Native)	One codebase for employee mobile app across Android/iOS, plus a web dashboard.	Middle ground if native feel is wanted without maintaining two native codebases.

The team should propose one option to the coordinator with reasoning, rather than defaulting silently — this decision affects the whole technology stack.

18. Technology Considerations

No technology stack is mandated by the client. The team should propose a stack and justify it against the requirements above (mobile capture, GPS, image upload, admin dashboard, export). Consider, at minimum:

- **Frontend (Employee):** mobile-responsive web (PWA) or native/cross-platform app — depends on Section 17 decision.
- **Frontend (Admin):** a standard web dashboard framework.
- **Backend:** a REST (or GraphQL) API layer with clear authentication and role checks.
- **Database:** a relational database is recommended given the structured, relational nature of employee/customer/record data.
- **File/Object Storage:** for site images, separate from the primary database.

- **Hosting:** cloud-hosted, containerised where practical, to simplify future scaling.

19. Database Design

PROPOSED / TO BE CONFIRMED

The following entities are a proposed starting schema, not a final one. Field names, data types and constraints must be refined by the team during requirement analysis.

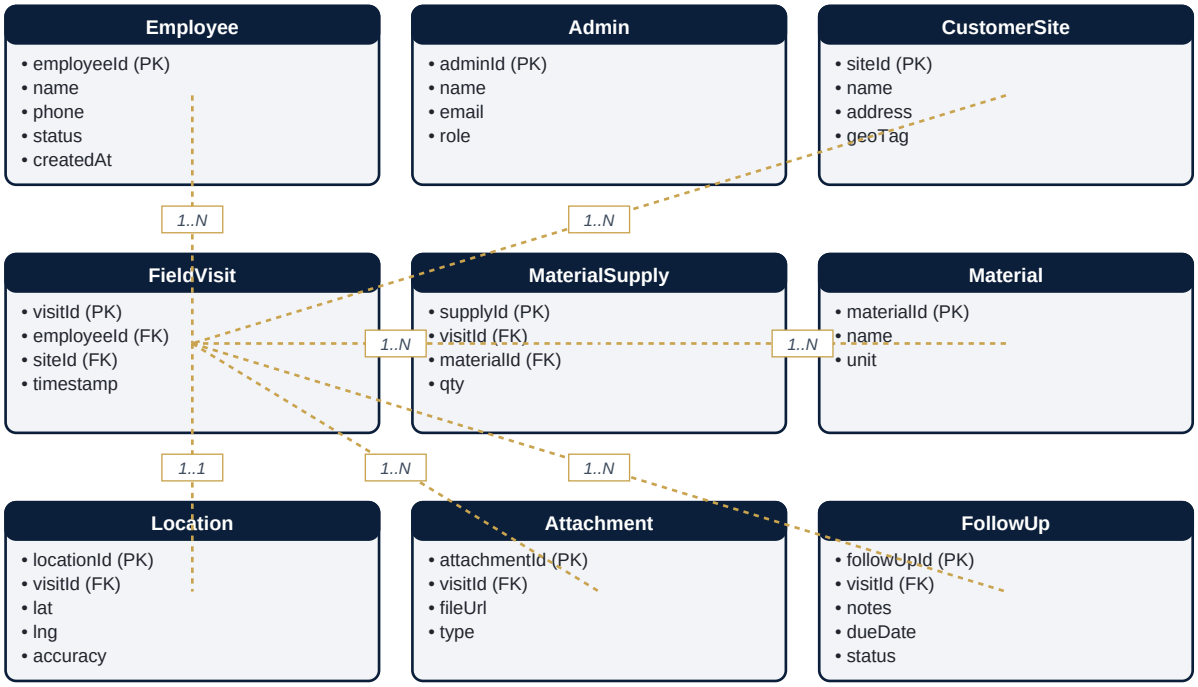

Fig 19.1 — Proposed entity relationships (subject to team refinement)

Relationship summary: one Employee creates many FieldVisit records; each FieldVisit belongs to one CustomerSite, has exactly one Location capture, may have multiple Attachments (images), may supply multiple Materials (via MaterialSupply), and may generate one or more FollowUp entries.

20. API Planning

Endpoint (indicative)	Method	Purpose
/auth/login	POST	Employee or Admin login, returns session/token.
/employees	GET / POST	Admin: list or create employees.
/employees/{id}	GET / PUT	Admin: view or update a specific employee.
/field-visits	GET / POST	List (admin, filterable) or create (employee) a field visit record.
/field-visits/{id}	GET	View full detail of a specific record.
/field-visits/{id}/attachments	POST	Upload an image against a record.
/materials	GET / POST	List or manage the material catalogue.
/exports	POST	Admin: generate a filtered export in the company template.

Exact endpoint names, request/response contracts, and versioning strategy (e.g. /api/v1/…) are left to the team to finalise during API design — this table is a planning starting point, not a frozen contract.

21. UI/UX Flow

Design for the field employee first: large touch targets, minimal required typing, camera and GPS actions front-and-centre, and a clear success/failure state after submission. The admin dashboard can follow conventional data-table + filter-panel + detail-drawer patterns.

- Employee: Login → Dashboard → New Entry → Capture (site, GPS, photo, material) → Review → Submit → Confirmation.
- Admin: Login → Dashboard (summary widgets) → Records table (search/filter) → Record detail (map + image + data) → Export.
- Wireframes are part of the Team Planning Exercise deliverable (Section 34) and are intentionally not pre-drawn here so the team owns the UI/UX design decisions.

22. Role-Based Access Control

Action	Employee	Administrator
Create own field record	Yes	No (not their role)
View own records	Yes	Yes (all)
View other employees' records	No	Yes
Manage employee roster	No	Yes
Generate exports	No	Yes

23. Security Considerations

- Authentication for both employee and admin logins, with password hashing (never plain text).
- Authorization enforced server-side on every endpoint — never trust client-side role checks alone.
- HTTPS everywhere; no sensitive data (location, images, credentials) over plain HTTP.
- Environment variables / secrets manager for API keys and database credentials — never commit secrets to Git.
- Secure, validated file uploads (type/size checks) to prevent malicious file upload.
- Audit logs for record creation and admin actions (who exported what, when).
- Rate limiting on login endpoints to reduce brute-force risk.

24. Validation & Error Handling

- Required fields (customer/site, material, quantity) validated both client-side (for fast feedback) and server-side (for integrity).
- GPS/image capture failures must produce a clear, actionable message — never a silent failure.
- Network failures during submission should not lose the employee's entered data; support local retry rather than forcing full re-entry.
- Admin-side filters should handle empty result sets gracefully with a clear 'no records found' state.

25. Testing Strategy

Fig 25.1 — Testing lifecycle for this project

Give particular attention to GPS-permission edge cases, image-upload failure/retry, and export correctness against whatever template the client ultimately confirms.

26. Deployment Strategy

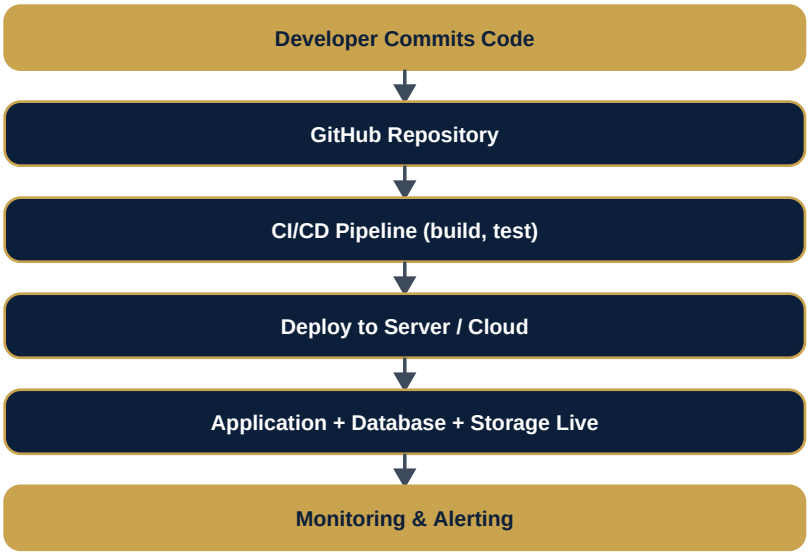

Fig 26.1 — Generic deployment pipeline

A specific hosting provider is not mandated — propose one during the Team Planning Exercise, with cost and ease-of-deployment as key factors for an internship-scale project.

27. Git/GitHub Workflow

Fig 27.1 — Branching and PR workflow

- Branches: **main** (production-ready), **develop** (integration), **feature/*** (per task).
- Use conventional, descriptive commit messages (e.g. feat:, fix:, docs:).
- Every change goes through a Pull Request with at least one reviewer before merging.
- Protect main and develop branches from direct pushes.
- Maintain a clear README, issue tracker usage, and up-to-date documentation as the project evolves.

28. Team Structure & Leadership Evaluation

The team decides its own internal structure — roles are not permanently pre-assigned in this handbook. Typical responsibilities to cover between team members:

- Team Lead / Coordinator
- Frontend Developer
- Backend Developer
- Database
- UI/UX

- Integration
- QA/Testing
- Documentation

A person may hold multiple responsibilities depending on team size; assign roles based on skills and interest, not by default.

Leadership evaluation criteria

Beyond the working product, this engagement will also be used to evaluate: initiative, communication, ownership, delegation, decision-making, conflict resolution, planning, technical leadership, accountability, and ability to deliver. Strong candidates may be considered for future Team Lead, Project Lead, Project Manager, or specialised technical responsibilities. This is an opportunity for growth, not a promise of promotion.

29. Suggested Task Breakdown & Development Milestones

Milestone	Indicative Scope
M1 — Foundations	Requirement analysis complete, architecture & stack decided, DB schema drafted, GitHub repo set up.
M2 — Core Backend	Auth, Employee management, Field Visit CRUD APIs, database migrations.
M3 — Field Employee App	Login, dashboard, new-entry flow with GPS + image capture, submission & validation.
M4 — Admin Dashboard	Employee management UI, records list with search/filter, record detail view.
M5 — Export & Reporting	Configurable export mapping, Excel/CSV generation, basic operational reports.
M6 — Hardening	Security review, validation/error-handling pass, performance checks.
M7 — Testing & UAT	Full test pass per Section 25, client/user acceptance testing, bug-fix cycle.
M8 — Deployment & Handover	Production deployment, documentation finalised, demo & presentation.

Exact sprint durations are left to the team's sprint plan (see Team Planning Exercise) based on team size and availability.

30. Documentation Requirements

- README with setup/run instructions.
- Architecture documentation (diagrams + rationale).
- Database documentation (schema + relationships).
- API documentation (endpoints, request/response formats).
- Test cases and test summary report.
- Deployment documentation (environments, steps, configuration).
- User guide (field employee) and Admin guide.

31. Deliverables & Acceptance Criteria

Deliverables

- Working application (employee + admin sides)
- Source code in a GitHub repository
- README
- Architecture, database and API documentation
- Test cases and test report
- Deployment details
- User/Admin documentation
- Final presentation and live demo

Acceptance criteria (indicative)

- A field employee can log in, create a record with GPS + image + material data, and see a confirmation, end to end.
- An administrator can search/filter records and successfully export a filtered set to a file.
- Role-based access is enforced — an employee cannot view another employee's records or reach admin functions.
- Core flows are covered by automated or documented manual tests, with results recorded.
- The application is deployed and reachable in a demo environment.

32. Future Scalability

- Design the database and APIs so additional record types (beyond field visits) can be added without a schema rewrite.
- Keep the export-mapping layer configurable so new company templates or additional export formats can be added later.
- Consider that employee count and record volume may grow significantly — avoid patterns that assume a small, fixed dataset (e.g. loading full tables into memory).
- Leave room for a future notifications/reminders module for pending follow-ups.

33. Open Questions to Clarify with Client

- What is the exact company export template (columns, format, naming) referenced in Section 13?
- Should field employees be able to edit or delete a record after submission, and if so, within what time window?
- Is continuous/background location tracking required, or only point-in-time capture at record creation?
- Is offline capture (no network at the site) a real-world requirement?
- What is the expected device platform for field employees — Android, iOS, or both?
- Is there an existing employee ID scheme in use elsewhere in the client's organisation that this must align with?
- Are there existing branding assets (logo, colours) the client wants reflected in the UI?
- What is the expected initial number of field employees and expected record volume per month?

34. Team Planning Exercise

Before development begins, the team must produce and submit for review:

- 1. Requirement analysis
- 2. User roles
- 3. Feature list
- 4. User flow
- 5. System architecture
- 6. Database / ER diagram
- 7. API list
- 8. UI wireframes
- 9. Technology stack proposal
- 10. GitHub strategy
- 11. Task breakdown
- 12. Sprint plan
- 13. Testing plan
- 14. Deployment plan

35. Final Delivery Checklist

Item	Status
Working application deployed and accessible	■
Source code pushed to GitHub with clean history	■
README complete and accurate	■
Architecture & database documentation complete	■
API documentation complete	■
Test cases and test report submitted	■
Deployment details documented	■
User & Admin guides complete	■
Final presentation & demo delivered	■
All open questions resolved or explicitly deferred with client sign-off	■

End of Project 1 Handbook — Version 1.0, Initial Project Brief.